

Architecture

How Jarvis is put together: its layers, modules, threads and data, and the outside services it relies on.

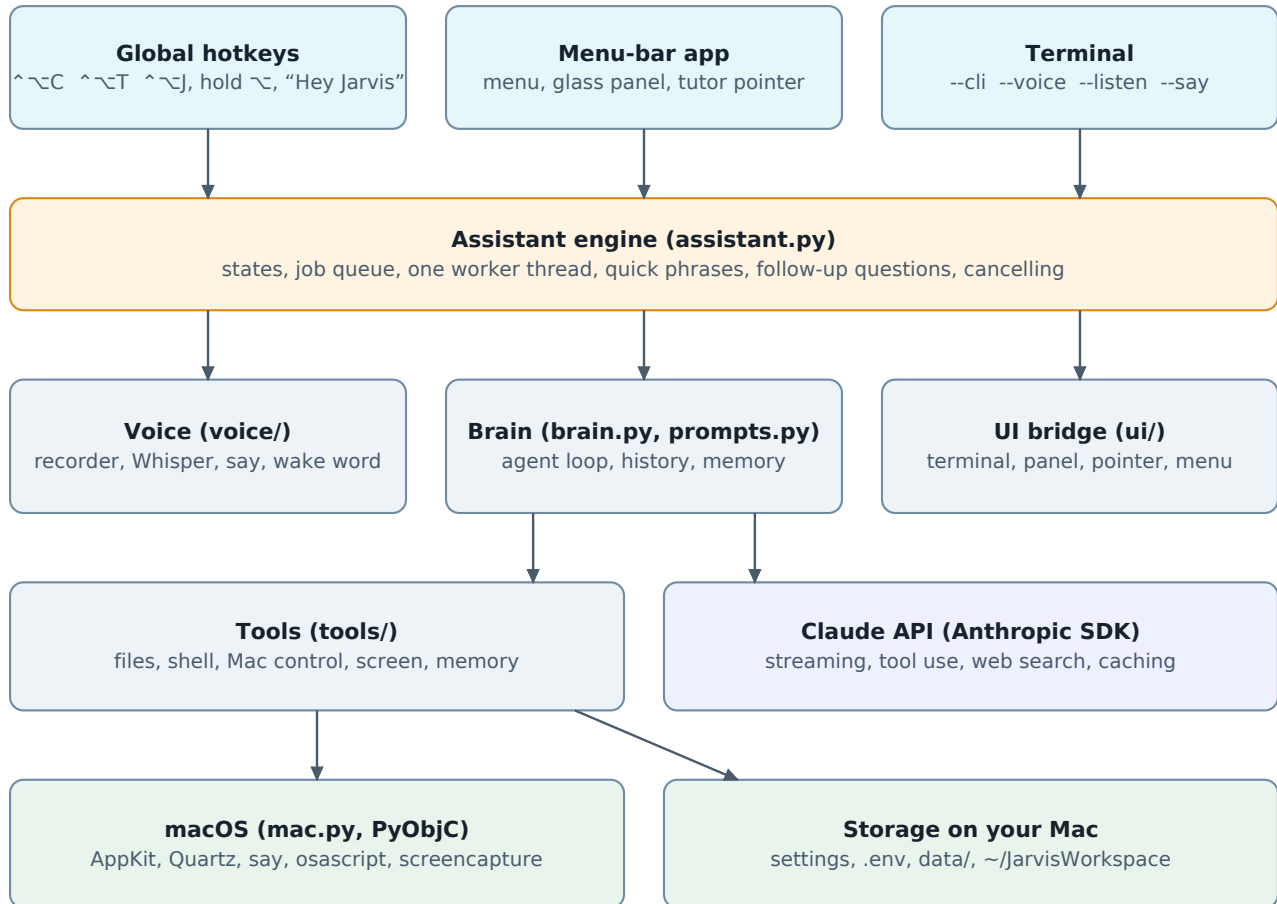
Jarvis 1.0, October 2026

In this document

- The big picture: what talks to what
- Modules and what each one owns
- The tools Claude can use
- Threads: how Jarvis stays responsive
- Where data lives
- Outside services and macOS features
- Key design decisions and their trade-offs

The big picture

Jarvis is a Python program that runs on your Mac in two forms: the **menu-bar app**, started with `./run.sh`, and the **terminal chat**, started with `./run.sh --cli`. Both share one core. Speech is recognised and spoken on the Mac itself; understanding, planning and choosing tools happen in Claude, reached through the Anthropic API.



Requests travel down from the ways in, through the assistant engine, to the brain and its tools. Everything you see travels back up through the UI bridge.

Every request takes the same path. Something wakes Jarvis (a hotkey, holding Option, the wake word, a menu click or a line typed in the terminal); the assistant engine turns it into a job; the brain sends it to Claude together with the tools that exist; the tools act on your Mac; and the reply is shown and spoken.

Jarvis needs macOS 12 or newer on an Intel or Apple Silicon Mac, and Python 3.11 or 3.12. Python 3.13 doesn't work on Intel Macs yet because `onnxruntime`, which the speech recogniser uses, has no Intel build for it.

Modules

All code lives in the `jarvis/` package. Each module has one job.

Module	What it owns	Phase
<code>__main__.py</code>	The command line: chooses the menu-bar app, terminal chat, microphone test, one question, doctor or start at login.	1
<code>config.py</code>	Settings: built-in defaults, then <code>config.yaml</code> , then <code>config.local.yaml</code> , then <code>JARVIS_MODEL</code> , then <code>--model</code> . Checks values, reports typos, sets up logging.	1
<code>claude_client.py</code>	Finds the API key, creates the Anthropic client, explains API errors in plain English.	1
<code>doctor.py</code>	<code>./run.sh --doctor</code> : system, packages, settings, permissions, hotkeys, microphone, speech, extras and the API.	1-10
<code>brain.py</code>	The agent loop: streams Claude's reply, runs its tool calls, keeps the conversation history small and valid.	2
<code>prompts.py</code>	Jarvis's standing instructions, built from the tools that exist, and the context block added to each request.	2
<code>memory.py</code>	Long-term facts about you, in <code>data/memory.json</code> .	2
<code>tools/</code>	What Jarvis can do (<code>file_tools</code> , <code>shell_tools</code> , <code>mac_tools</code> , <code>screen_tools</code> , <code>memory_tools</code>) and the registry that runs them.	2-9
<code>cli.py</code>	Terminal chat, the microphone test and one-shot questions.	2-9
<code>mac.py</code>	macOS helpers: AppleScript, dialogs, clipboard, synthetic key presses, screenshots, permission checks.	4
<code>voice/recorder.py</code>	Microphone capture with voice detection.	5
<code>voice/stt.py</code>	Speech to text with faster-whisper, on the Mac.	5
<code>voice/tts.py</code>	Spoken replies with the macOS <code>say</code> command.	6
<code>assistant.py</code>	The engine: states, job queue and worker thread, quick phrases, follow-up questions, cancelling.	6
<code>ui/__init__.py</code>	The UI bridge interface and the terminal UI (<code>ConsoleUI</code>).	2
<code>ui/app.py</code>	The menu-bar app: status icon, menu, hotkey wiring, dialogs.	7
<code>ui/hotkeys.py</code>	Global hotkeys and hold-to-talk, through a Quartz event tap.	7
<code>ui/menu_panel.py</code>	The status line in the menu, the notification fallback and the panel group.	7-8
<code>ui/hud.py</code> , <code>ui/hud_logic.py</code>	The floating glass panel and its orb: AppKit drawing, and plain-Python geometry.	8
<code>ui/overlay.py</code> , <code>ui/overlay_logic.py</code>	The tutor pointer: AppKit drawing, and plain-Python geometry.	9
<code>voice/wakeword.py</code>	The optional “Hey Jarvis” wake word, with <code>openWakeWord</code> .	10
<code>login.py</code>	Start at login, through a Login Item that runs <code>Jarvis.command</code> in Terminal.	10

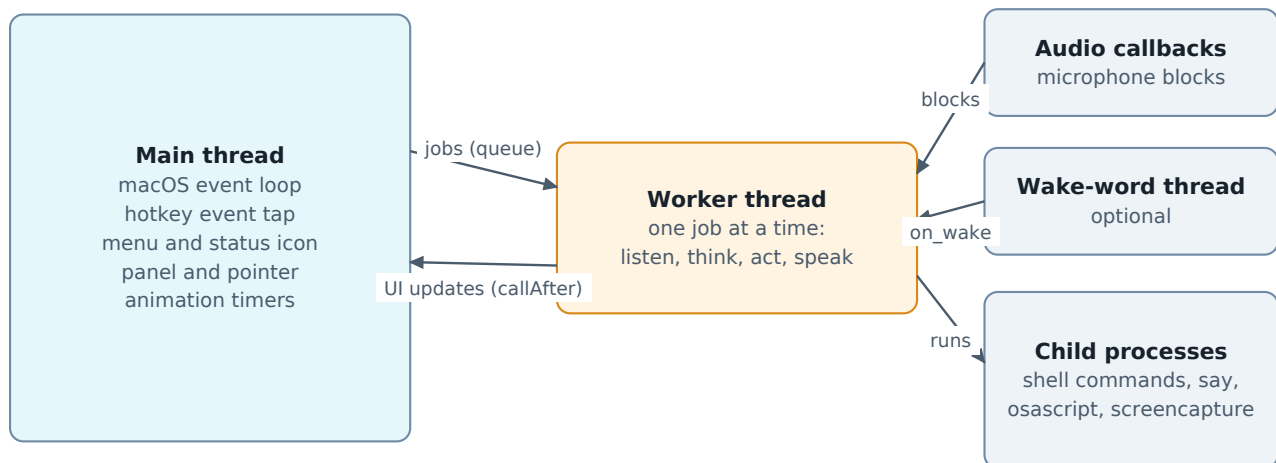
The tools Claude can use

Group	Tools	What they do
Files	<code>read_file</code> <code>write_file</code> <code>edit_file</code> <code>list_dir</code>	Read code, text, PDFs, Word documents and images; create, append to and edit files; list folders.
Shell	<code>run_shell</code>	Run commands in zsh with a timeout, or in the background for servers (output in <code>data/logs/background/</code>).
Mac control	<code>open</code> <code>run_applescript</code> <code>clipboard</code> <code>type_text</code> <code>get_selected_text</code> <code>frontmost_app</code> <code>notify</code> <code>speak</code>	Open apps, sites and files; control apps; copy and paste; type into the app in front; read your selection; notify; speak during a task.
Screen	<code>take_screenshot</code> <code>point_at</code> <code>hide_pointer</code>	See the main display and point at things with the tutor pointer.
Memory	<code>memory</code>	Add, remove and list long-term facts about you.
Web	<code>web_search</code>	Runs at Anthropic (tool version <code>web_search_20250305</code>), up to 5 searches per request.

The prompt only describes tools that are installed, so Jarvis never claims abilities it doesn't have. In phase 2, for example, it only knew about memory and web search, and said so when asked to edit a file.

Threads

Jarvis keeps hotkeys and animations smooth by never doing slow work on the main thread.



- **Main thread.** Runs the macOS event loop (`AppHelper.runEventLoop`): the hotkey event tap, the menu, the panel, the pointer and their animation timers. Handlers here only change state or queue a job.
- **Worker thread.** Takes jobs (listen, typed request, text) from a queue one at a time, so requests never overlap.
- **Audio callbacks.** PortAudio delivers microphone blocks on its own thread; the recorder reads them from a queue.

- **Wake-word thread.** Optional. Listens for “Hey Jarvis” and steps aside while Jarvis uses the microphone.
- **Child processes.** Shell commands (each in its own process group, so they can be stopped as a whole), `say`, `osascript` for AppleScript and dialogs, `screencapture` and `afplay` for the chimes.

Every screen update from the worker goes through the **UI bridge**. In the app that is `AppUIBridge`, which hands each call to the main thread with `AppHelper.callAfter`; in the terminal it is `ConsoleUI`, which prints.

Where data lives

Path	What it holds	In git
<code>config.yaml</code>	Every setting with its default and an explanation. Not for editing.	Yes
<code>config.local.yaml</code>	Your own settings; they override <code>config.yaml</code> .	No
<code>.env</code>	Your API key (<code>ANTHROPIC_API_KEY</code>) and, optionally, <code>JARVIS_MODEL</code> .	No
<code>data/logs/jarvis.log</code>	The log. It rotates at 1 MB and keeps 3 older files.	No
<code>data/logs/background/</code>	Output of servers and long jobs started in the background.	No
<code>data/models/</code>	The Whisper speech model (about 480 MB for <code>small.en</code>).	No
<code>data/memory.json</code>	Long-term memory: up to 200 facts.	No
<code>data/backups/</code>	Copies of files outside the workspace, taken before Jarvis changed them.	No
<code>~/JarvisWorkspace/</code>	Where Jarvis creates projects and documents unless you name another place.	n/a
<code>patches/</code>	Update kits and the backups they make.	No
<code>Jarvis.command</code>	Created by <code>./run.sh --login</code> on to start Jarvis at login.	No
<code>.venv/</code>	The Python environment <code>./setup.sh</code> creates.	No

Outside services and macOS features

Dependency	Used for
Anthropic API (anthropic SDK 1.x)	Claude: streaming replies, tool use, web search, prompt caching. Default model <code>claude-sonnet-5-5</code> .
faster-whisper with CTranslate2	Speech recognition on the processor with 8-bit weights; model <code>small.en</code> by default. Its voice filter uses <code>onnxruntime</code> .
sounddevice (PortAudio)	Microphone input.
PyObjC: AppKit, Quartz, ApplicationServices	Menu bar, panel, pointer, event-tap hotkeys, synthetic key presses, permission checks.
openWakeWord (optional)	The “Hey Jarvis” wake word, with an ONNX model.
macOS commands	<code>say</code> (speech), <code>osascript</code> (AppleScript and dialogs), <code>screencapture</code> , <code>pbcopy</code> and <code>pbpaste</code> , <code>open</code> , <code>afplay</code> , <code>textutil</code> (Word and RTF files), <code>sips</code> (HEIC images).
pypdf, Pillow, NumPy, PyYAML, python-dotenv	PDF text, image resizing, audio maths, settings files and the <code>.env</code> file.

Key design decisions

Decision	Why	Trade-off
Python with PyObjC, no Electron or Qt	Native menu bar, panel and event taps in one language; light on an Intel Mac.	The AppKit code only runs on a Mac, so tests use stand-ins for it.
Speech recognition on the Mac	Private, free and quick: your voice never leaves the Mac.	A one-time download of about 480 MB; accuracy depends on model size.
say for speech	Built in, offline, interruptible, many voices.	Less natural than cloud voices.
Runs inside Terminal	macOS permissions belong to Terminal and are granted once, for the app and the chat alike.	A Terminal window stays open, including when started at login.
Event tap for hotkeys	Catches <code>^⌘C</code> anywhere and keeps the key from reaching the app you're in.	Needs Accessibility permission.
Prompt built from installed tools	Jarvis never claims abilities it doesn't have.	Tool guidance lives in <code>prompts.py</code> , keyed by tool name.
Prompt caching and history trimming	Cheaper and faster replies in long sessions.	Very old details are shortened; files can be read again.
One worker thread	Requests never overlap or race each other.	A long task holds up the next request until it ends or you press Esc.
Graceful fallbacks	If the panel or pointer can't start, Jarvis keeps working through the menu bar and notifications.	The fallback shows less.
Patch kits for updates	One command applies, tidies, tests and commits.	Kits are built for this repository's history.